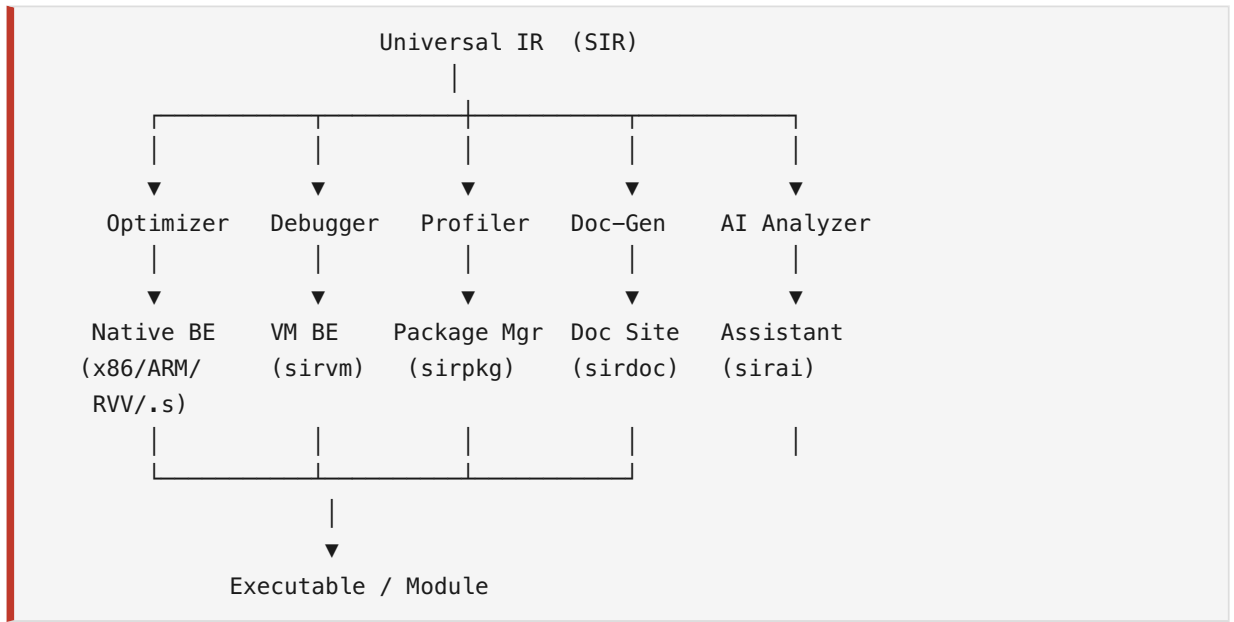


SIR — Sanskrit IR (Universal Intermediate Representation)

संस्कृत IR — a universal, machine-code-level Intermediate Representation written in Sanskrit (Romanized / Hinglish) words. SIR is the single hub that every front end lowers INTO and every back end lowers OUT OF.



SIR is the **one assembly everyone can write**. Any front end (Sakum, C, Rust, Python, a DSL, or a future language) emits SIR; SIR then fans out to native machine code, a portable VM, a packaged module, live docs, and an AI analyzer.

1. SIR word model (Sanskrit → Hinglish → meaning)

Every SIR op is a Sanskrit keyword (Romanized). ASCII/Hinglish aliases are allowed so anyone can type them. Mnemonics:

SIR op (Sanskrit)	Hinglish alias	Meaning
stha	load	load immediate / from memory into reg
rakh	store	store reg into memory
yoj	add	add
viyoj	sub	subtract
gun	mul	multiply

SIR op (Sanskrit)	Hinglish alias	Meaning
bhag	div	divide
shesh	mod	modulo
sam	eq	equal compare
chot	lt	less-than compare
kud	jmp	unconditional jump
yadi	if	conditional jump (if flag)
bula	call	call node/function
laut	ret	return
labh	phi	SSA phi (merge)
chhan	alloc	stack/heap alloc
mudra	print	print (debug/observe)
shant	nop	no-op
ant	halt	halt

Registers: `r0 ... r15` (virtual; the native back end maps them to the ISA).

Flags: `f0` (zero), `f1` (sign). Comments: `# ...` or `// ...`.

2. SIR program shape

```
@node add_two          # a node = a function/IR unit (callable)
  stha r0 2             # r0 = 2
  stha r1 3             # r1 = 3
  yoj r2 r0 r1          # r2 = r0 + r1
  mudra r2              # print r2
  laut r2               # return r2
@end
```

A SIR module is a bag of `@node`s plus a `@entry` marker.

3. The `|>` connector (Universal Link / Search)

`|>` is the **special connector keyword**. It does two jobs:

- Node linking (auto-selection by user):** connect the output of one node to the input of another. The compiler auto-selects the target node by name, or the user may explicitly name it.

```
@node main stha r0 10 r0 |> double # pipe r0 into node `double`, result back into r0 r0 |
> square |> mudra @end
```

1. **Search (functions + IR):** `|>` with a query token searches the IR space for a matching node or function and binds it.

```
r0 |> ?double # search IR for any node named *double*, link it r0 |> ?"vector add" #
search by description / tag
```

The `sir` tool answers `?` queries by scanning loaded modules' `@node` tables and tags (`@tag`). This makes SIR a **searchable universal IR**: write once, find and rewire anywhere.

4. Back ends (all lower FROM SIR)

- **Native (sirc -native)** → raw x86-64 `.intel_syntax` asm (honors the doctrine: emits real machine-level code). ARM64 / RVV planned.
- **VM (sirc -vm)** → portable `sirvm` bytecode (stack-ish, for embedded/unsafe hosts). Runs under `tools/sirvm.py`.
- **Package (sirpkg)** → bundles `@node`s + tags into a versioned module.
- **Doc (sirdoc)** → renders each `@node` to markdown/diagram.
- **AI (sirai)** → analyzes a node, suggests optimizations, explains IR.

5. Tool chain

<code>sir</code>	– driver: assemble / compile / run / search / link
<code>sirc</code>	– the SIR assembler+optimizer (front of all back ends)
<code>sirvm</code>	– the portable VM back end
<code>siropt</code>	– optimizer pass (constant-fold, dead-code, peephole)
<code>sirdbg</code>	– debugger (step nodes, inspect <code>r0..r15</code> , <code>f0/f1</code>)
<code>sirprof</code>	– profiler (per-node cycle estimate + call count)
<code>sirdoc</code>	– documentation generator
<code>sirpkg</code>	– package manager
<code>sirai</code>	– AI assistant analyzer

See `tools/sir/README.md` for the command reference. SIR is living: the ब्रम्ह bot can emit SIR nodes and link them with `|>` automatically.